

Exploring Design Patterns: Facade and Flyweight

Eazdan Mostafa Rafin

Roll Number: 2003055

April 7, 2025

1 Introduction

In the world of software development, design patterns act like best practices or blueprints that guide developers to solve common problems in a structured way. Among the many patterns available, two particularly useful ones—especially in terms of simplifying code and improving performance—are the **Facade Pattern** and the **Flyweight Pattern**.

This report dives into these two patterns with real-life analogies, C++ implementations, and explanations designed to not only inform, but also make the concepts intuitive and approachable.

2 Pattern 1: Facade

2.1 What is the Facade Pattern?

The **Facade Pattern** provides a simplified interface to a complex subsystem. It allows clients to interact with multiple classes in a system through a single, unified interface — effectively hiding the complexity of the underlying implementation.

2.2 Real-Life Analogy

Smart Home System:

Imagine living in a smart home. Before leaving the house, we usually do several things:

- Turn off all the lights
- Set the thermostat to an energy-saving temperature
- Lock the doors and activate the security system

- Turn off the music

Doing all of this manually can be time-consuming and involves interacting with several different systems, each with its own interface.

Now, imagine your smart home app has a single “**Leave Home**” button. When you press it, all these tasks are performed automatically in the background.

This single button acts as a **facade** — it provides a simplified interface to control multiple complex subsystems.

Key Idea: The *Facade Design Pattern* is like that one button — it hides the complexity of multiple systems and provides an easy-to-use, unified interface.

2.3 Visual Example

A consumer calls one number and speaks with a customer service representative. The customer service representative acts as a Facade, providing an interface to the order fulfillment department, the billing department, and the shipping department.

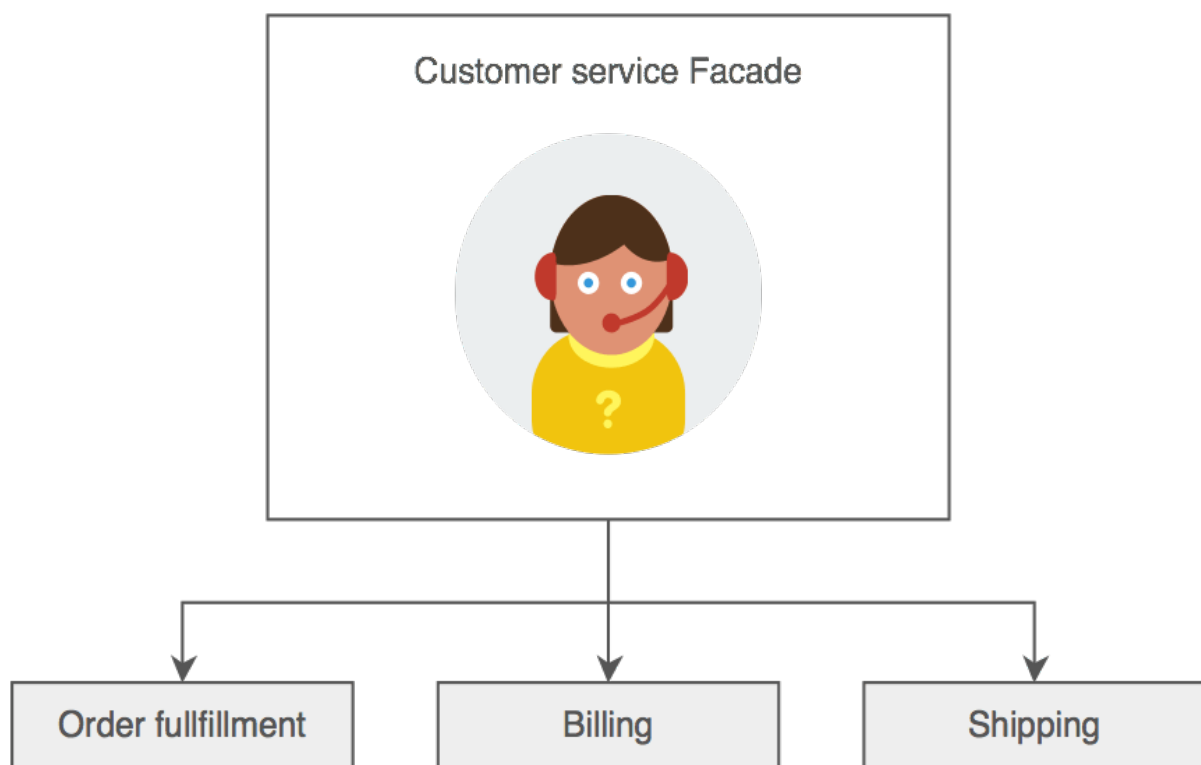


Figure 1: Facade

2.4 C++ Example: Home Theater System

```
#include <iostream>
```

```

using namespace std;

class DVDPlayer {
public:
    void on() {
        cout << "DVD Player is now ON" << endl;
    }
    void play() {
        cout << "Playing the DVD..." << endl;
    }
};

class Projector {
public:
    void on() {
        cout << "Projector is warming up..." << endl;
    }
};

class SoundSystem {
public:
    void on() {
        cout << "Sound system is set to cinema mode" << endl;
    }
};

class HomeTheaterFacade {
private:
    DVDPlayer dvd;
    Projector projector;
    SoundSystem sound;
public:
    void watchMovie() {
        cout << "Initializing home theater setup..." << endl;
        dvd.on();
        projector.on();
        sound.on();
        dvd.play();
    }
};

// Main
int main() {
    HomeTheaterFacade homeTheater;
    homeTheater.watchMovie();
    return 0;
}

```

Listing 1: Facade Pattern in a Home Theater System (C++)

3 Pattern 2: Flyweight

3.1 What is the Flyweight Pattern?

The **Flyweight Pattern** is used to reduce memory consumption when working with a large number of similar objects. It separates the intrinsic (shared) state from the extrinsic (unique) state and reuses objects that share the same intrinsic data.

3.2 Real-Life Analogy

Text Editor Characters:

Imagine we're using a text editor like Microsoft Word or Google Docs. As we type a long document, we end up with thousands of characters—letters, numbers, and symbols.

Each character may have formatting such as:

- Font family (e.g., Arial, Times New Roman)
- Font size
- Style (bold, italic)
- Color

Storing every single character with all of its formatting would consume a large amount of memory, especially in longer documents.

To solve this, the text editor uses the **Flyweight Design Pattern**.

How it works:

- A shared object is created for each unique character format.
- The system reuses this shared object every time that specific style is needed.
- Only the unique information (like the character's position in the document) is stored separately.

For instance, if we have 5,000 occurrences of the letter “e” in the same font and style, the system only keeps one shared object instead of 5,000.

Key Idea: The Flyweight Pattern allows us to efficiently manage memory by sharing common data across many objects, while keeping only the unique details separate.

3.3 Visual Example

When browser loads a web page, it traverse through all images on that page. Browser loads all new images from Internet and places them the internal cache. For already loaded images, a flyweight object is created, which has some unique data like position within the page, but everything else is referenced to the cached one.

Browser loads images
just once and then
reuses them from pool:



Figure 2: Flyweight

3.4 C++ Example: Game Enemy Objects

```
#include <iostream>
#include <map>
#include <memory>
using namespace std;

class EnemyFlyweight {
private:
    string type;
    int health;
    int attack;
    int defense;
    string texture;
public:
    EnemyFlyweight(string t, int h, int a, int d, string tex)
        : type(t), health(h), attack(a), defense(d), texture(tex) {}

    void displaySharedProperties() {
        cout << type << ": HP=" << health
            << ", ATK=" << attack
```

```

        << " , DEF=" << defense
        << " , TEX=" << texture << endl;
    }
};

class EnemyFactory {
private:
    map<string, shared_ptr<EnemyFlyweight>> flyweights;
public:
    shared_ptr<EnemyFlyweight> getEnemy(string type, int h, int a, int
    d, string tex) {
        string key = type + tex;
        if (flyweights.find(key) == flyweights.end()) {
            cout << "Creating new Flyweight for " << type << endl;
            flyweights[key] = make_shared<EnemyFlyweight>(type, h, a, d
            , tex);
        }
        return flyweights[key];
    }
};

class Enemy {
private:
    shared_ptr<EnemyFlyweight> flyweight;
    pair<int, int> position;
    int currentHealth;
public:
    Enemy(shared_ptr<EnemyFlyweight> fw, pair<int, int> pos, int ch)
        : flyweight(fw), position(pos), currentHealth(ch) {}

    void render() {
        flyweight->displaySharedProperties();
        cout << "Position: (" << position.first << "," << position.
            second
            << "), Current HP: " << currentHealth << "\n" << endl;
    }
};

// Main
int main() {
    EnemyFactory factory;

    auto orc = factory.getEnemy("Orc", 100, 15, 5, "orc_texture.png");
    auto goblin = factory.getEnemy("Goblin", 50, 10, 2, "goblin_texture
    .png");

    Enemy enemies[] = {
        Enemy(orc, {10, 20}, 80),
        Enemy(orc, {15, 25}, 60),
        Enemy(goblin, {5, 10}, 40),

```

```
        Enemy(goblin, {7, 12}, 30)
    };
    for (auto& enemy : enemies) {
        enemy.render();
    }

    return 0;
}
```

Listing 2: Flyweight Pattern in a Game Enemy System (C++)

4 Conclusion

Both the **Facade** and **Flyweight** design patterns offer elegant solutions to different kinds of software design problems. The Facade pattern excels at simplifying complex systems by exposing a single interface, while the Flyweight pattern focuses on performance, making it ideal for applications where object instantiation needs to be minimized.

By using these patterns wisely, developers can write cleaner, more maintainable, and memory-efficient code — all while maintaining flexibility and scalability in their software architecture.